Spojové seznamy

DELTA - Střední škola informatiky a ekonomie, s.r.o.

Ing. Luboš Zápotočný

19.03.2026

CC BY-NC-SA 4.0

Abstraktní datové struktury

Co jsou abstraktní datové struktury?

Abstraktní datová struktura (ADT) definuje **co** má datová struktura dělat, ne **jak** to má dělat

- Rozšiřují možnosti programového řešení
- Reprezentace dat, které v jazyce nativně neexistují
- Pro člověka srozumitelnější práce s daty

Příklady ADT

- **Seznamy** (Lists)
- Zásobníky (Stacks)
- Fronty (Queues)
- Binární stromy (Binary trees)
- Grafy (Graphs)
- …

Dnes se zaměříme na **seznamy** – konkrétně **spojové seznamy**

Spojový seznam

Motivace – proč ne pole?

Co když chceme **často vkládat a mazat** prvky uprostřed?

10	20	30	40	50	60

Vložení prvku 25 na 3. pozici $\Rightarrow$ musíme **posunout** všechny následující prvky

10	20	**25**	30	40	50	60

Složitost této operace je $\mathcal{O}(n)$, co ostranění prvku uprostřed? $\mathcal{O}(n)$

Pole vs. spojový seznam

Pole (array):

- Prvky jsou uloženy **za sebou v paměti**
- Přístup k prvku přes index: $\mathcal{O}(1)$
- Vložení/smazání uprostřed: $\mathcal{O}(n)$ (posun prvků)

Spojový seznam (linked list):

- Prvky mohou být **kdekoliv v paměti**
- Propojeny pomocí **ukazatelů**
- Vložení/smazání: $\mathcal{O}(1)$ (pokud známe pozici)
- Přístup k i-tému prvku: $\mathcal{O}(n)$ (musíme projít seznam)

Jak to vypadá v paměti?

Pole – prvky leží za sebou:

Adresa	0x100	0x104	0x108	0x10C	0x110
Hodnota	9	6	27	42	15

Jak to vypadá v paměti?

Spojový seznam – prvky mohou být rozptýleny:

Adresa	0x200	0x350	0x420	0x510	0x780
Data	9	6	15	27	42
Next	0x350	0x510	NULL	0x780	0x420

Každý prvek si **pamatuje**, kde je ten následující

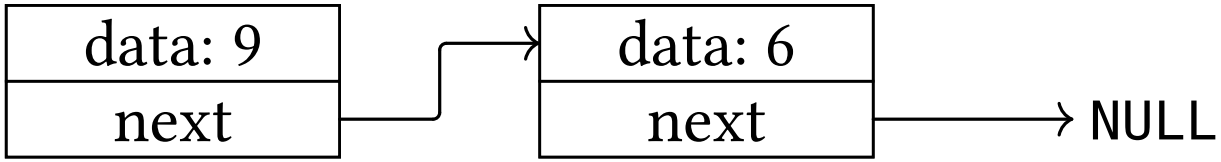
Struktura Node

Node – stavební blok seznamu

Každý prvek spojového seznamu se nazývá **Node** (uzel)

Obsahuje:

- **data** – samotná hodnota (int, char, pointer, ...)
- **next** – ukazatel na **následující** Node v seznamu
- Poslední Node má `next == NULL`



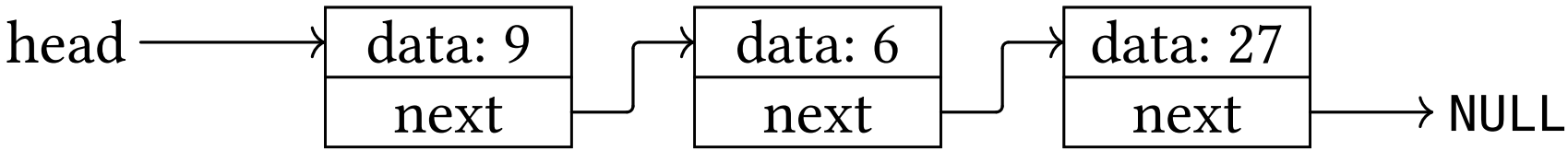
Definice v jazyce C

```c
typedef struct Node {
    int data;
    struct Node *next;
} Node;
```

Proč `struct Node *next` a ne `Node *next`?

V okamžiku definice `next` ještě **neexistuje** alias `Node` (ten se vytváří až na konci `typedef`) ⇒ musíme použít plné jméno `struct Node`

Jednosměrně zřetězený seznam



- **head** – ukazatel na první Node v seznamu
- Procházíme vždy **od head** směrem k poslednímu prvku
- Poslední prvek poznáme podle `next == NULL`

Vytvoření seznamu

```
Node *head = (Node *) malloc(sizeof(Node));
Node *second = (Node *) malloc(sizeof(Node));
Node *third = (Node *) malloc(sizeof(Node));

head->data = 9;
head->next = second;

second->data = 6;
second->next = third;

third->data = 27;
third->next = NULL;
```

Vytvoření seznamu

Důležité: v praxi nikdy nevytváříme seznam takto ručně – používáme pomocné funkce, které si ale musíme naimplementovat sami

Výpis seznamu

Princip procházení (traversal):

1. Začneme u `head`
2. Zpracujeme aktuální Node
3. Přejdeme na `current->next`
4. Opakujeme, dokud nenarazíme na `NULL`

Tento vzor se opakuje u **všech** operací nad seznamem

Implementace print_list v C

```c
void print_list(Node *head) {
    Node *current = head;
    while (current != NULL) {
        printf("%d -> ", current->data);
        current = current->next;
    }
    printf("NULL\n");
}
```

Výstup: `9 -> 6 -> 27 -> NULL`

Operace nad spojovým seznamem

Vložení na začátek (prepend)

Chceme vložit **4** na začátek seznamu `9 -> 6 -> 27`

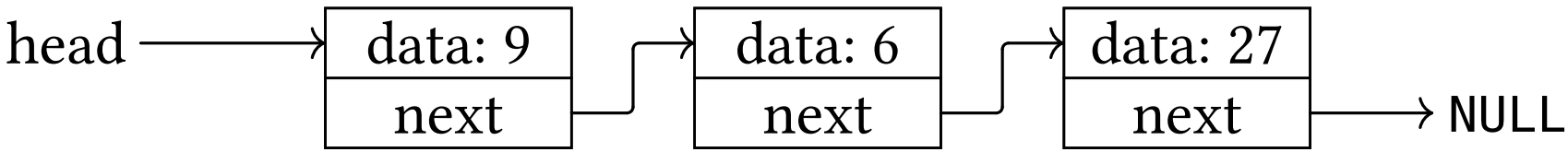
Postup:

1. Vytvoříme nový Node s hodnotou 4
2. Nastavíme `new_node->next = *head`
3. Přesuneme `head` na nový Node

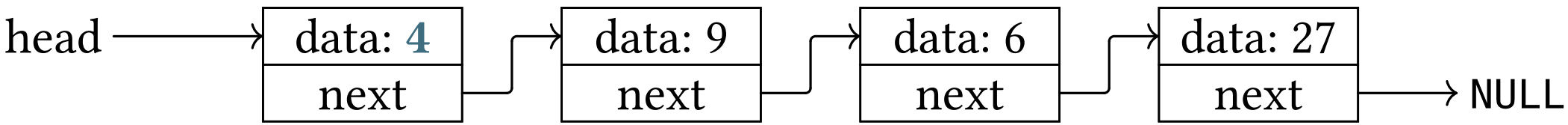
Výsledek: `4 -> 9 -> 6 -> 27`

Vložení na začátek (prepend)

Před:



Po:



Pointer na pointer – proč `Node **head`?

Než se pustíme do kódu, musíme pochopit důležitý koncept

Funkce, které **mění head** (přidávání/mazání prvního prvku), potřebují `Node **head`

Proč?

- `head` je pointer na první Node
- Chceme-li **změnit** kam `head` ukazuje, potřebujeme **pointer na tento pointer**
- Stejný princip jako u pomocných funkcí pro pole

Implementace prepend v C

```
// This does NOT work - head is local copy
void prepend_wrong(Node *head, int data) {
    Node *new_node = (Node *) malloc(sizeof(Node));
    new_node->data = data;
    new_node->next = head;
    head = new_node;  // Only changes local copy!
}
```

Implementace prepend v C

```
// This works - we modify the original pointer
void prepend(Node **head, int data) {
    Node *new_node = (Node *) malloc(sizeof(Node));
    new_node->data = data;
    new_node->next = *head;
    *head = new_node;  // Changes the original pointer
}
```

Složitost: $\mathcal{O}(1)$ – neprocházíme žádný prvek seznamu

Vložení na konec (append)

Chceme vložit **42** na konec seznamu `9 -> 6 -> 27`

Postup:

1. Vytvoříme nový Node s hodnotou 42
2. Projdeme seznam **až na konec** (najdeme Node s `next == NULL`)
3. Nastavíme `last->next = new_node`

Výsledek: `9 -> 6 -> 27 -> 42`

Implementace append v C

```
void append(Node **head, int data) {
    Node *new_node = (Node *) malloc(sizeof(Node));
    new_node->data = data;
    new_node->next = NULL;

    if (*head == NULL) { *head = new_node; return; }

    Node *current = *head;
    while (current->next != NULL) {
        current = current->next;
    }
    current->next = new_node;
}
```

Implementace append v C

Proč `if (*head == NULL)`?

Pokud je seznam **prázdný**, nemáme žádný Node, ke kterému bychom připojili nový $\Rightarrow$ nový Node se stává `head`

Složitost: $\mathcal{O}(n)$ – musíme projít celý seznam až na konec

Vložení za konkrétní prvek (insert after)

Chceme vložit **15** za Node s hodnotou 6: `9 -> 6 -> 27`

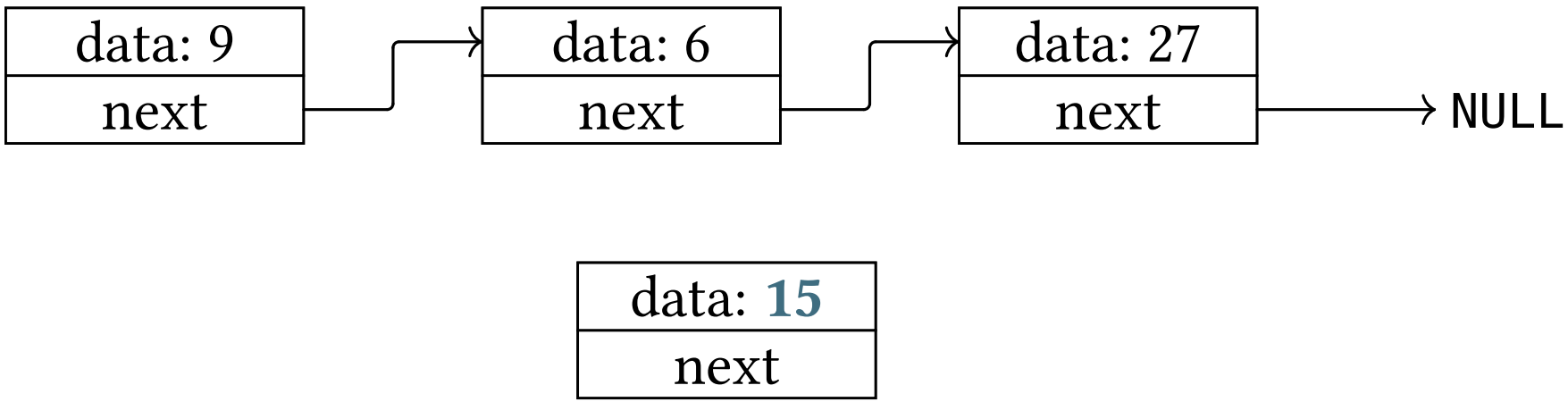
Postup:

1. Vytvoříme nový Node s hodnotou 15
2. Napojíme `new_node->next = prev->next`
3. Přepojíme `prev->next = new_node`

Výsledek: `9 -> 6 -> 15 -> 27`

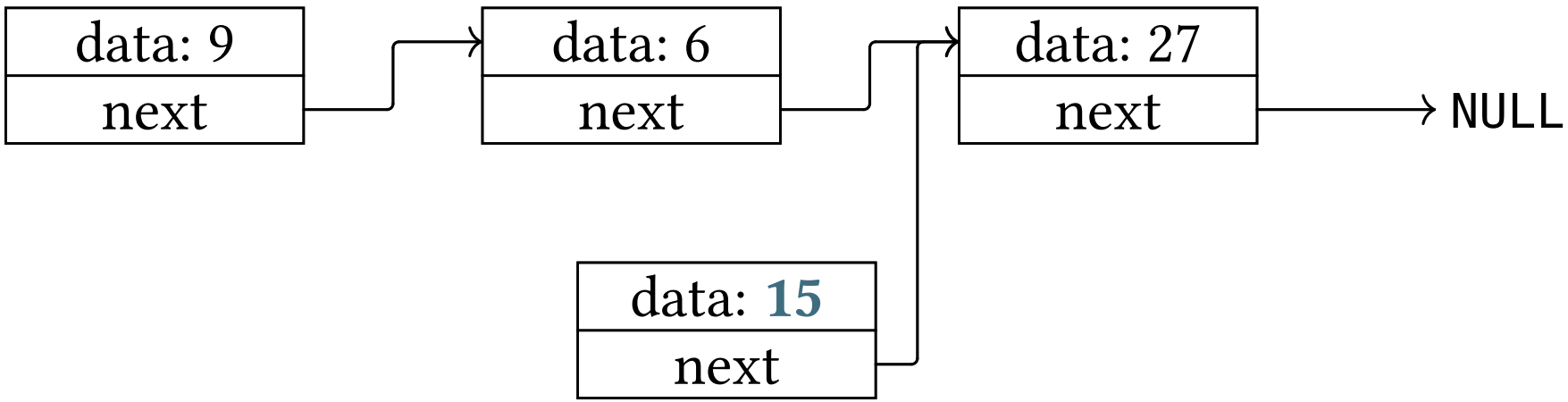
Vložení za konkrétní prvek (insert after)

Krok 1 – vytvoříme nový Node:



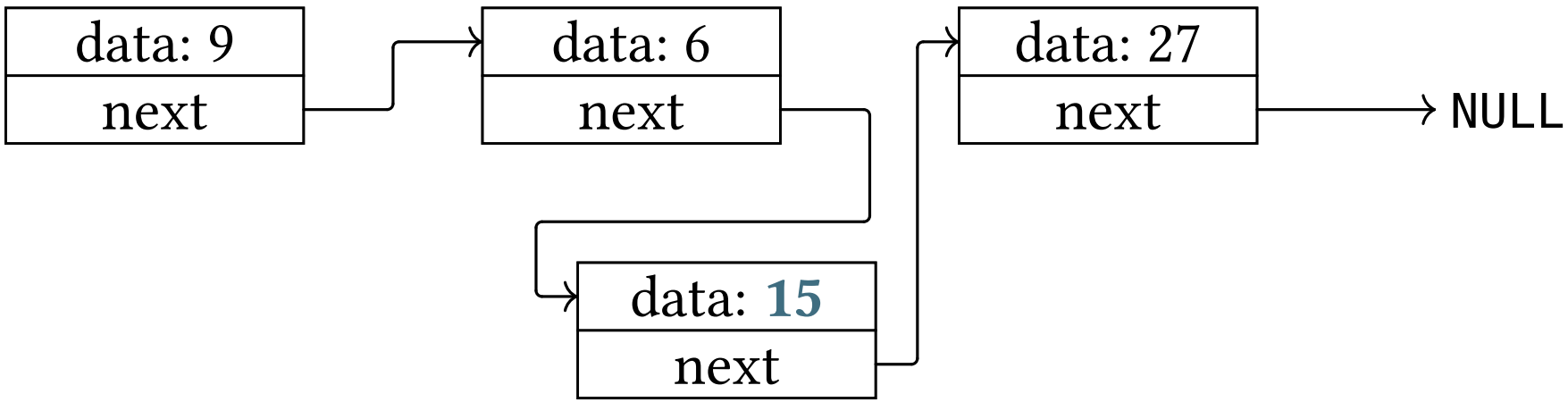
Vložení za konkrétní prvek (insert after)

Krok 2 – `new_node->next = prev->next` (nový Node ukazuje na 27):



Vložení za konkrétní prvek (insert after)

Krok 3 – `prev->next = new_node` (6 ukazuje na nový Node):



Implementace insert_after v C

```
void insert_after(Node *prev, int data) {
    if (prev == NULL) {
        return;
    }

    Node *new_node = (Node *) malloc(sizeof(Node));
    new_node->data = data;
    new_node->next = prev->next;
    prev->next = new_node;
}
```

Implementace insert_after v C

Důležité: pořadí operací je klíčové! Kdybychom **nejdříve** přepojili `prev->next`, ztratili bychom odkaz na zbytek seznamu

Složitost:

- `find(head, 6)` pak `insert_after(...)` – hledáme `prev`: $\mathcal{O}(n)$
- Máme-li `prev` přímo (z předchozí operace): $\mathcal{O}(1)$

Vyhledání prvku

Chceme najít Node s konkrétní hodnotou

```
Node *find(Node *head, int data) {
    Node *current = head;
    while (current != NULL) {
        if (current->data == data) {
            return current;
        }
        current = current->next;
    }
    return NULL;  // Not found
}
```

Vyhledání prvku

Složitost: $\mathcal{O}(n)$ – v nejhorším případě projdeme celý seznam

Smazání prvku

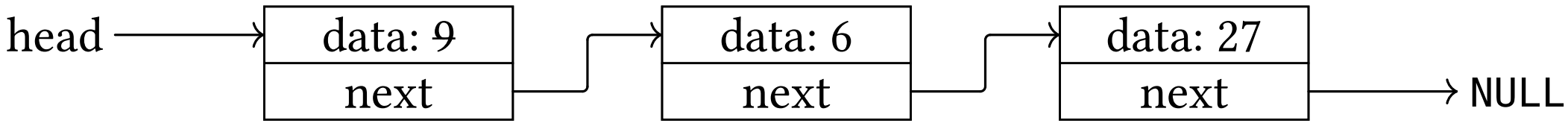
Smazání je složitější, protože musíme rozlišit dva případy:

1. **Mazání head**: musíme přesunout `head` pointer na další Node
2. **Mazání uprostřed/konce**: najdeme Node **před** mazaným a přepojíme

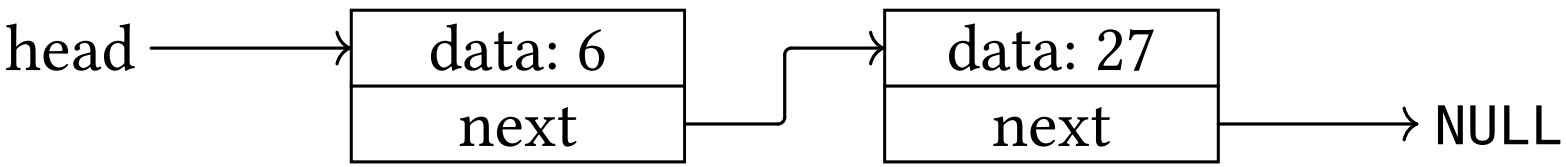
V obou případech **musíme uvolnit paměť** pomocí `free()`!

Smazání prvku

Mazání head (hodnota 9): `9 -> 6 -> 27`

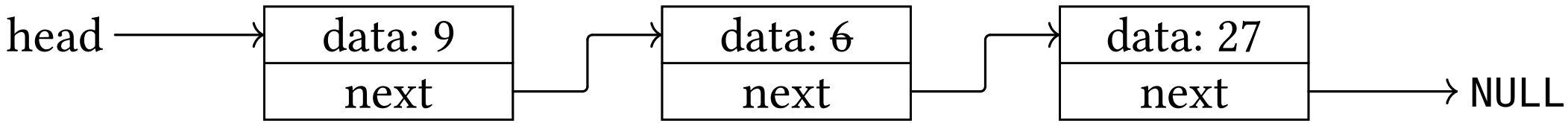


⇒ Přesuneme `head` a uvolníme starý Node:

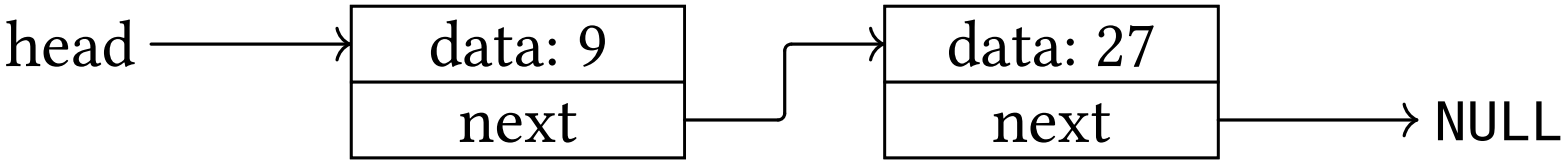


Smazání prvku

Mazání uprostřed (hodnota 6): `9 -> 6 -> 27`



⇒ Přepojíme `9->next` na `27` a uvolníme starý Node:



Implementace delete v C

```c
void delete(Node **head, int data) {
    if (*head == NULL) return;          // Empty list

    if ((*head)->data == data) {        // Deleting head
        Node *temp = *head;             // Save old head
        *head = (*head)->next;          // Move head forward
        free(temp);                     // Free memory
        return;
    }
```

Implementace delete v C

```
    // Find predecessor and remove node
    Node *current = *head;
    while (current->next != NULL) {
        if (current->next->data == data) {
            Node *temp = current->next;
            current->next = temp->next;
            free(temp);
            return;
        }
        current = current->next;
    }
}
```

Implementace delete v C

Proč potřebujeme `temp`?

```
// WRONG - memory leak!
*head = (*head)->next;
// We lost the pointer to the old head - cannot free it!

// CORRECT
Node *temp = *head;        // Save pointer to old head
*head = (*head)->next;     // Move head forward
free(temp);                // Free the old head
```

Bez `temp` bychom ztratili odkaz na starý Node a nemohli ho uvolnit ⇒ **memory leak**

Uvolnění celého seznamu

Když seznam již nepotřebujeme, musíme uvolnit **všechny** Nody

```
void free_list(Node **head) {
    Node *current = *head;
    while (current != NULL) {
        Node *temp = current;
        current = current->next;
        free(temp);
    }
    *head = NULL;
}
```

Uvolnění celého seznamu

Pozor: musíme si nejdříve uložit `current->next`, teprve potom `free(current)` – jinak ztratíme přístup ke zbytku seznamu

Složitost operací

Přehled složitostí

Operace	Pole	Spoj. seznam	Spoj. seznam (známe pozici)
Přístup k i-tému prvku	$\mathcal{O}(1)$	$\mathcal{O}(n)$	$\mathcal{O}(1)$
Vložení na začátek	$\mathcal{O}(n)$	$\mathcal{O}(1)$	$\mathcal{O}(1)$
Vložení na konec	$\mathcal{O}(1)$	$\mathcal{O}(n)$	$\mathcal{O}(1)$

Přehled složitostí

Operace	Pole	Spoj. seznam	Spoj. seznam (známe pozici)
Vložení uprostřed	$\mathcal{O}(n)$	$\mathcal{O}(n)$	$\mathcal{O}(1)$
Smazání	$\mathcal{O}(n)$	$\mathcal{O}(n)$	$\mathcal{O}(1)$
Vyhledání	$\mathcal{O}(n)$	$\mathcal{O}(n)$	$\mathcal{O}(n)$

Přehled složitostí

Kdy použít pole?

- Potřebujeme **rychlý přístup** k libovolnému prvku
- Data se mění zřídka
- Chceme **paměťovou lokalitu** (cache-friendly)

Kdy použít spojový seznam?

- Často **vkládáme a mažeme** prvky
- Nepotřebujeme přístup přes index
- Nevíme předem, kolik prvků budeme mít

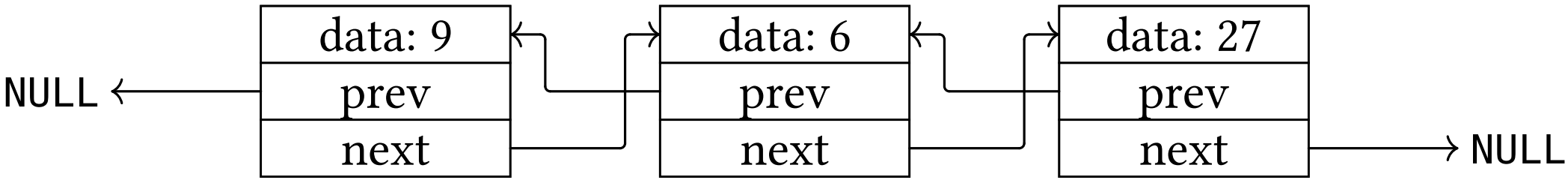
Varianty spojových seznamů

Obousměrně zřetězený seznam

Každý Node má navíc ukazatel **prev** na předchozí prvek

```
typedef struct DNode {
    int data;
    struct DNode *prev;
    struct DNode *next;
} DNode;
```

Obousměrně zřetězený seznam



Výhody:

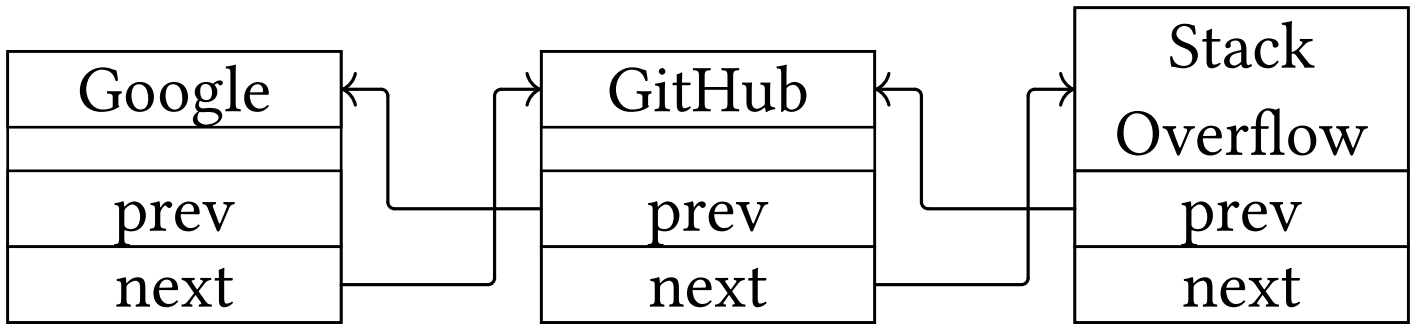
- Procházení **oběma směry**
- Smazání Node **bez znalosti** předchůdce – stačí `node->prev`
- Vložení **před** konkrétní Node je jednoduché

Nevýhoda: každý Node zabírá víc paměti (extra pointer)

Obousměrně zřetězený seznam

Příklady použití:

Historie prohlížeče (zpět/vpřed):



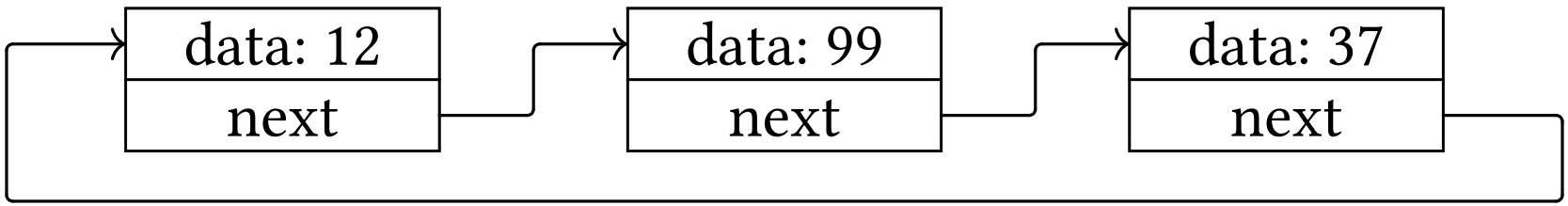
- Tlačítko **zpět** = `current = current->prev`
- Tlačítko **vpřed** = `current = current->next`

Obousměrně zřetězený seznam

Undo/redo v textových editorech funguje na stejném principu

Kruhově zřetězený seznam

Poslední Node ukazuje **zpět na head** (místo na NULL)



Pozor: při procházení nemůžeme testovat `next == NULL` – musíme si pamatovat, kde jsme začali!

Kruhově zřetězený seznam

Použití:

- **Round-robin scheduling** – operační systém přiděluje čas procesům dokola
- Cyklické buffery
- Multiplayer hry – střídání hráčů

Lze rozšířit na **obousměrně kruhově** zřetězený seznam (každý Node má `prev` i `next`, oba konce propojené)

Praktické použití

Kde se spojové seznamy používají?

- Implementace **front** (FIFO) a **zásobníků** (LIFO)
- Historie v prohlížeči (zpět/vpřed)
- Round-robin scheduling v OS
- Reprezentace **grafů** (seznamy sousedů)
- Správa paměti v OS (free list – seznam volných bloků)
- Linux kernel – `vm_area_struct` pro správu virtuální paměti

Kompletní příklad

```
int main() {
    Node *head = NULL;  // Empty list

    // Build the list
    append(&head, 9);
    append(&head, 6);
    append(&head, 27);
    print_list(head);   // 9 -> 6 -> 27 -> NULL

    // Insert at the beginning
    prepend(&head, 4);
    print_list(head);   // 4 -> 9 -> 6 -> 27 -> NULL
```

Kompletní příklad

```
    // Find and insert after
    Node *six = find(head, 6);
    insert_after(six, 15);
    print_list(head);   // 4 -> 9 -> 6 -> 15 -> 27 -> NULL

    // Delete
    delete(&head, 9);
    print_list(head);   // 4 -> 6 -> 15 -> 27 -> NULL

    // Clean up
    free_list(&head);
    return 0;
}
```

Srovnání

Pole vs. spojový seznam – shrnutí

	Pole	Spojový seznam
Paměťová lokalita	Dobrá (cache)	Špatná
Velikost	Fixní / dynamická	Dynamická
Extra paměť	Žádná	Pointer na node
Náhodný přístup	Ano – $\mathcal{O}(1)$	Ne – $\mathcal{O}(n)$
Vkládání/mazání	Pomalé – $\mathcal{O}(n)$	Rychlé – $\mathcal{O}(1)$

Pole vs. spojový seznam – shrnutí

Paměťová lokalita – proč na ní záleží?

Moderní procesory mají **cache** paměť, která uchovává bloky dat z RAM

- **Pole**: prvky leží za sebou ⇒ při načtení jednoho se do cache dostanou i sousední
- **Spojový seznam**: prvky jsou rozptýleny ⇒ každý přístup může být **cache miss**

V praxi bývá pole **rychlejší**, i když má horší asymptotickou složitost pro vkládání